A Simulation-Based Mission Computer and Resilient Communications Platform: Architecture, Fault Injection, and Evaluation

Tyree Franklin Jr.
Computer Science Student
Western Governors University
https://github.com/tyreefranklinjr

***Abstract*—This paper examines a mission-computer simulation and a separate resilient communications simulator implemented in C++20, C11, and Rust. The mission computer combines synthetic inertial and satellite-navigation measurements, a four-state Kalman estimator, grid-based A* planning, proportional-integral-derivative command generation, and watchdog-driven state management. The communications simulator implements sequenced messages, acknowledgement timeouts, bounded transmission attempts, duplicate suppression, and score-based link failover. Evaluation combines source inspection with numerical analysis of committed artifacts at a fixed repository revision. All 500 recorded fault scenarios satisfy the implemented final-state checks. Across 1,500 trajectory samples, planar position root-mean-square error is 0.138 m. Twenty communications trials record simulated failover latencies of 22–41 ms, with a mean of 39 ms. Source inspection also identifies limits on these findings: mission-state safing does not inhibit actuator commands, controller output does not drive the simulated plant, and random-number scaling changes the intended communications loss model. The study documents the implemented behavior, quantifies the archived results, and defines verification criteria for integrated and hardware-based evaluation.**

***Index Terms*—Mission computers, software-in-the-loop simulation, fault injection, Kalman filtering, A* search, watchdogs, communications resilience.**

I. Introduction

Mission-oriented software coordinates estimation, planning, command generation, and health supervision. Its response to a fault depends on both the local algorithm and the interfaces between these functions. A useful prototype must make those interfaces inspectable and associate its claims with specific evidence.

The Mission-Computer–Communications-Platform repository implements these concerns in two executables [1]. The C++ mission computer represents four logical nodes within one process. A separate Rust executable examines message delivery during a primary-link outage.

This study asks three questions:

1) Do the committed fault scenarios produce the mission states required by the implemented checks?
2) What estimation errors and link-switching latencies appear in the committed artifacts?
3) Which implementation and experimental limits prevent stronger system-level conclusions?

The contribution is an engineering case study rather than a new estimation or search algorithm. It connects implementation details to measured outputs and identifies the additional evidence needed to evaluate integrated operation.

II. Scope and Evidence

A. Revision and Review Method

The evaluated revision is:

e04681fdf181b30d5230861b9ff7cf7fef1f30c1

The review covers the mission-computer source, Rust communications source, two C++ test programs, build configuration, trial script, and four committed CSV files [2], [3], [4], [5].

The evidence has three distinct forms. Source inspection establishes what is implemented. Calculations over committed CSV files establish what those artifacts record. Proposed changes and acceptance criteria describe work that has not been performed in this study.

The executables were not rebuilt or rerun for this manuscript. The numerical findings are therefore a reanalysis of archived outputs, not an independent reproduction of execution.

B. Execution Boundary

Both executables run as ordinary software processes. The implementation does not use physical sensors, a CAN transceiver, a radio, or an ESP32-C3. FreeRTOS is not linked.

The mission model advances in 20 ms logical steps; the communications model advances in 1 ms logical steps. These increments define simulated time. They do not measure operating-system scheduling accuracy or hardware deadlines.

The executables do not exchange operational messages. Their outputs support separate subsystem observations, not an integrated communications-and-control result.

III. System Architecture

A. Mission-Computer Execution

Every mission tick invokes the sensor, navigation, control, and supervisor functions in that order. Table I summarizes their responsibilities.

Sequential execution simplifies inspection and scenario generation. It does not represent preemption, concurrent tasks,

TABLE I
LOGICAL MISSION-COMPUTER NODES

Node	Responsibility	Output
Sensor	Scripted motion and noisy measurements	IMU/GNSS frames and heartbeat
Navigation	State estimation and route planning	Position estimate and heartbeat
Control	Waypoint-directed command calculation	Actuator frame and heartbeat
Supervisor	Health checks and state transitions	Mission state

independent clocks, or a scheduler that remains responsive when a real task blocks.

The injected system-overrun fault skips control-node execution. It models missing work and heartbeats rather than excessive processor consumption.

B. C/C++ Messaging

The C library implements a 64-entry ring buffer of CAN-style frames. Each frame contains an identifier, priority field, payload length, and eight-byte payload. Publication and reception take constant time. Storage is fixed, and rejected publications increment a drop counter.

Reception follows FIFO order. The priority field does not implement CAN arbitration. Serialization time, physical errors, bus arbitration, and bus-off recovery are outside the model.

Floating-point values are packed using memory copies. A deployment across heterogeneous processors would need an explicit byte order and numeric representation.

C. Mission-State Management

The state machine contains five states: `INIT`, `CALIBRATING`, `AUTONOMOUS`, `DEGRADED`, and `SAFE_STOP`.

Initialization enters calibration. Completion of the calibration interval permits autonomous operation. A communications outage moves autonomous operation into the degraded state; restored communications permits recovery. An unhealthy core-system indication selects `SAFE_STOP`, which remains terminal for that run.

This state selection does not gate the control function. The implementation can continue generating commands after entering `SAFE_STOP`. The demonstrated property is therefore terminal state selection, not actuation inhibition.

IV. ALGORITHMIC DESIGN

A. State Estimation

The estimator maintains position and velocity:

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & v_{x,k} & v_{y,k} \end{bmatrix}^T. \tag{1}$$

For acceleration input $\mathbf{u}_k = [a_x, a_y]^T$, prediction is

$$\hat{\mathbf{x}}_k^- = F\hat{\mathbf{x}}_{k-1} + B\mathbf{u}_k, \tag{2}$$

with

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix}. \tag{3}$$

GNSS measures position:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \tag{4}$$

The covariance prediction, gain, and state correction are

$$P_k^- = FP_{k-1}F^T + Q, \tag{5}$$

$$K_k = P_k^- H^T (HP_k^- H^T + R)^{-1}, \tag{6}$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + K_k(\mathbf{z}_k - H\hat{\mathbf{x}}_k^-). \tag{7}$$

The implementation uses $Q = 0.02I_4$, $R = 0.5I_2$, and

$$P_k = (I - K_k H)P_k^-. \tag{8}$$

Although the source class is named `Ekf`, its transition and measurement models are linear. A four-state linear Kalman estimator is the precise description of the implemented mathematics.

Fixed-size matrices give constant work per update for this state dimension. Further evaluation should check covariance symmetry, positive semidefiniteness, and behavior under ill-conditioned measurement updates.

The simulated GNSS noise is uniform with amplitude 0.5 m. Its ideal per-axis variance is $0.5^2/3 \approx 0.0833\ \mathrm{m}^2$. The filter instead uses a measurement-covariance entry of 0.5. The tuning is therefore not a direct match to the simulator's noise variance.

B. Grid-Based Planning

The planner uses four-connected, unit-cost moves and

$$f(n) = g(n) + h(n), \tag{9}$$

where

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}. \tag{10}$$

Euclidean distance is a consistent lower bound under these movement rules. The search follows the A* formulation introduced by Hart, Nilsson, and Raphael [6].

The mission instance uses a 20×20 grid with a 2 m coordinate increment. Replanning is requested every 250 ticks, or 5 s of simulated time, and when no path has been established.

The open list is a vector. Selecting and removing its lowest-score entry requires linear work in the frontier size. On this bounded-degree grid, worst-case search cost can grow quadratically with the number of cells.

The occupancy array holds 4,096 cells, but constructor arguments are not checked against that capacity. A reusable version should reject invalid dimensions before initializing the array.

C. Command Generation

The controller applies proportional, integral, and derivative terms with output saturation:

$$u_k = \mathrm{clip}\left(K_p e_k + K_i I_k + K_d D_k, -2, 2\right). \tag{11}$$

The gains are $K_p = 1.2$, $K_i = 0.02$, and $K_d = 0.15$. The integral is clamped to $[-4, 4]$.

The controller input is

$$e_k = \mathrm{atan2}(y_{\mathrm{wp}} - \hat{y}_k,\, x_{\mathrm{wp}} - \hat{x}_k). \tag{12}$$

This is a waypoint bearing. The code does not subtract vehicle heading, so the input is not a conventional heading-tracking error. Controller output also does not alter the scripted trajectory. The current evidence supports command calculation, not closed-loop tracking or stability.

D. Communications

The Rust simulator uses a priority heap, an in-flight message map, a receiver sequence set, and a rolling link-success history [3].

New messages are generated every 10 simulated milliseconds. Successful attempts have a 5 ms acknowledgement delay; unsuccessful attempts time out after 20 ms. Each message receives at most five total transmission attempts. The constant named `MAX_RETRIES` includes the initial attempt in this limit.

The primary link is forced down over $[5000, 35000)$ ms. For $m \leq 6$ resolved outcomes, its score is

$$S = \frac{1}{m}\sum_{i=1}^{m} d_i, \qquad d_i \in \{0, 1\}. \tag{13}$$

Failover requires at least three recorded outcomes and a primary score below 0.5.

The heap sends more urgent messages first and uses older sequence numbers to break ties. However, the simulator drains the entire outgoing queue in one iteration. Without a service-capacity constraint, this ordering does not establish priority-dependent latency under congestion.

V. Evaluation and Results

A. Fault Outcomes

Each recorded scenario represents 500 mission ticks, or 10 simulated seconds, with one injected fault. Table II reports the recalculated outcomes.

All 500 final-state checks pass [4]. Of these, 311 require `SAFE_STOP`; the other 189 require avoiding that state. This is not a result of 500 successful safe stops.

For tolerated faults, the existing oracle accepts either `AUTONOMOUS` or `DEGRADED` as the final state. It does not separately require evidence of both degradation and recovery.

Heartbeat health fails when its age exceeds five ticks. The recorded five-tick reaction interval starts at the first omitted heartbeat. The last successful heartbeat occurred one tick earlier, so heartbeat age is six ticks at detection.

TABLE II
Committed Fault-Sweep Results

Category	Cases	Pass	Stop delay
Network outage	125	125	N/A
Skipped control task	125	125	100 ms
CAN publication fault	125	125	100 ms
Short GNSS dropout	64	64	N/A
Long GNSS dropout	61	61	800 ms
Total	500	500	

Delays use simulated time and are measured from the injection tick. N/A denotes a case that should not enter the terminal stop state.

TABLE III
Position Error in the Committed Trajectory

Metric	Value
Planar RMSE	0.1379 m
Mean planar error	0.1224 m
95th percentile, nearest-rank	0.2412 m
Maximum planar error	0.3621 m
Samples below 0.3 m	1,490 / 1,500
Fraction below 0.3 m	99.33%

GNSS safing occurs on the forty-first consecutive missing sample. The recorded interval from the first missing-sample tick is 40 ticks.

The cases establish conformance within the generated scenarios. They do not estimate operational reliability or cover simultaneous faults.

B. Estimation Accuracy

For each committed trajectory sample, planar error is

$$e_k = \sqrt{(\hat{x}_k - x_k)^2 + (\hat{y}_k - y_k)^2}. \tag{14}$$

The planar root-mean-square error is

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_k^2}. \tag{15}$$

Table III uses all 1,500 samples, including initialization.

CSV positions are rounded to four decimal places. The measurements describe one scripted trajectory and noise realization. They do not establish performance under multipath, inertial bias, asynchronous sampling, or real sensor outages.

C. Link Switching

Switching latency is

$$L_{\mathrm{switch}} = t_{\mathrm{switch}} - t_{\mathrm{outage}}. \tag{16}$$

Table IV summarizes the 20 committed trials.

These are virtual-time measurements. Switching the selected link is distinct from restoring successful application-level delivery.

The seed-7 event log records 4,394 transmission attempts, 4,000 unique-delivery events labeled `acked`, 393 timeouts, and 140 suppressed duplicates. Its single failover occurs at 5,041 ms. No failback event appears.

TABLE IV
SIMULATED LINK-SWITCHING LATENCY

Metric	Value
Trials	20
Minimum	22 ms
Mean	39 ms
Median	41 ms
Maximum	41 ms
Trials below 250 ms	20 / 20

A new message is transmitted at the terminal timestamp, 40,000 ms. Its normal acknowledgement would occur after execution ends. Delivery accounting must distinguish such unresolved messages from losses.

VI. THREATS TO VALIDITY

A. Probability Model

The Rust generator extracts `state » 33`, retaining 31 bits, but divides by the maximum 32-bit unsigned value. Its output therefore occupies approximately $[0, 0.5)$ rather than $[0, 1)$.

Under a uniform-state approximation, configured loss thresholds of 0.03 and 0.05 correspond to approximately 6% and 10% event probabilities. The late-arrival threshold of 0.20 corresponds to approximately 40%. Priority proportions also change.

The recorded results remain observations of the committed implementation. They do not validate its intended probability configuration. Correcting the generator requires regenerating the communications baseline.

B. Repeatability and Recovery

A fixed seed alone does not ensure identical event traces. Iteration over randomized `HashMap` storage can change the order in which resolved outcomes enter the rolling history. Deterministic replay should sort due events before applying them.

Failback requires the primary score to exceed 0.9. The inactive primary receives no explicit health probes, so restoring its availability does not itself provide new successful outcomes. Recovery requires an explicit probing policy and tests.

C. Safety and Integration

Terminal mission-state selection does not establish a safe physical effect. Actuator commands are not inhibited, the plant does not respond to those commands, and the Rust simulator does not transport mission traffic.

The current artifacts cannot establish closed-loop control performance, integrated command continuity, or physical stopping behavior.

D. Test Coverage

The existing tests cover estimator convergence and small-grid planning [5]. They do not establish complete state-transition coverage, malformed-frame handling, queue saturation behavior, or bounded communications memory use.

The planner test checks an upper bound on path length and verifies obstacle avoidance. It does not explicitly check adjacency between consecutive cells. A stronger oracle should verify each transition and compare path cost with an independent shortest-path reference.

VII. VERIFICATION ROADMAP

The next evaluation should address correctness before expanding the operating envelope.

A. Immediate Gates

1) **Actuation inhibition:** Define a maximum delay from fault detection to command inhibition. Check that nonzero commands remain inhibited until an explicit reset.
2) **Probability correction:** Verify the generator's range and event frequencies. Regenerate every communications artifact after correcting the scaling.
3) **Stronger fault checks:** Verify the complete transition sequence, reaction deadline, final state, and actuator behavior for each scenario.

B. Integration Gates

1) **Closed-loop plant:** Feed commands into an explicit plant model, include vehicle heading in the tracking error, and measure route error and stopping behavior.
2) **Operational interface:** Exchange sequenced mission messages between the executables. Measure loss, duplication, stale delivery, and application-level outage duration.
3) **Recovery and resource bounds:** Test primary-link probing, repeated outage cycles, queue saturation, outstanding-message limits, and duplicate-history expiration.

C. Target Evaluation

Hardware testing should record execution times, jitter, deadline misses, and watchdog behavior under load. Each result should identify its commit, configuration, toolchain, target, and input seeds.

A comparative communications experiment should run fixed-primary transmission, score-based failover, and probe-assisted recovery under identical offered loads and outage schedules. Endpoints should include unique delivery ratio, application-level outage duration, latency percentiles, unresolved messages, and peak queue occupancy.

VIII. REPRODUCIBILITY

The following commands describe the repository workflow. They were inspected but not executed for this manuscript.

```
git clone https://github.com/tyreefranklinjr/Mission-
     Computer--Communications-Platform.git
cd Mission-Computer--Communications-Platform
git checkout e04681fdf181b30d5230861b9ff7cf7fef1f30c1

cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
```

```
mkdir -p evaluation
./build/mission_computer sim \
  evaluation/trajectory.csv
./build/mission_computer fault-sweep \
  500 evaluation/faults.csv

cd comms_link
cargo build --release --locked
bash run_failover_trials.sh
```

The Rust trial script writes into `samples/` and updates the seed-7 event log. Reproduction should use a disposable checkout to preserve the archived baseline.

An execution record should retain compiler versions, operating system, build flags, command outputs, raw CSV files, and analysis procedures.

IX. Conclusion

The platform implements estimation, grid search, message exchange, mission-state supervision, and link-failover simulation in C++, C, and Rust. Its committed artifacts contain 500 satisfied final-state checks, a planar position RMSE of 0.138 m on one trajectory, and simulated switching latencies of 22–41 ms across 20 trials.

These results have a defined scope. The current implementation selects a stop state without inhibiting actuation, calculates commands without closing the plant loop, and evaluates communications separately from mission traffic. The probability-model defect also requires a new communications baseline.

The repository provides an inspectable foundation for further experiments. Connecting safety decisions to actuation, correcting the probability model, and strengthening the test oracles are the next steps toward a defensible integrated evaluation.

References

[1] T. Franklin Jr., "Mission-Computer–Communications-Platform," GitHub repository, revision e04681fdf181b30d5230861b9ff7cf7fef1f30c1 , accessed Oct. 20, 2026. [Online]. Available: https://github.com/tyreefranklinjr/Mission-Computer--Communications-Platform/tree/e04681fdf181b30d5230861b9ff7cf7fef1f30c1

[2] T. Franklin Jr., "Mission-computer implementation," `src/`, repository revision specified in [1]. Files include `main.cpp`, `ekf.hpp`, `astar.hpp`, `pid.hpp`, `watchdog.hpp`, `mission_state.hpp`, `fault_injector.hpp`, `imu_gnss_sim.hpp`, and `can_bus.c/.h`.

[3] T. Franklin Jr., "Communications simulator," `comms_link/src/main.rs`, `comms_link/src/link.rs`, and `comms_link/run_failover_trials.sh`, repository revision specified in [1].

[4] T. Franklin Jr., "Committed simulation artifacts," `samples/`, repository revision specified in [1]. Files: `fault_sweep_results.csv`, `ekf_trajectory.csv`, `comms_link_failover_latencies.csv`, and `comms_link_log.csv`.

[5] T. Franklin Jr., "Estimator and planner tests," `tests/test_ekf.cpp` and `tests/test_astar.cpp`, repository revision specified in [1].

[6] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968, doi: 10.1109/TSSC.1968.300136.